

目 录

第一章 本课题主要工作	4
第一节 机械手结构建模	4
第二节 机械手运动学分析与仿真	4
第三节 机械手结构强度校核与有限元分析	5
第四节 Matlab 工具箱函数仿真	5
第二章 Simulink 子课题主要工作	7
第一节 设计基础	7
1.1 Simulink 设计基础	7
第二节 确定 DH 参数	7
第三节 正运动学推导及 simulink 设计	8
第四节 逆运动学推导及 simulink 设计	10
4.1 计算 θ_1	11
4.2 计算 θ_3	11
4.3 计算 θ_2	12
4.4 计算 θ_5	12
4.5 计算 θ_4 和 θ_6	12
4.6 simulink 设计	13
第五节 关节位置梯型加减速及 simulink 设计	14
第六节 simulink 导入机械臂模型	19
第七节 simulink 轨迹模块设计	21
第八节 控制实物运行	23
第三章 设计总结	25
第一节 课程设计背景与目的	25
第二节 课程设计主要内容与实施过程	25
2.1 机器人系统认知与前期准备	25
2.2 半实物仿真系统设计与实现	25
3.3 半实物控制系统设计与调试	26
第三节 课程设计收获与体会	26

3.1 专业知识的深化与拓展	26
3.2 工程实践能力的提升	27

第一章 本课题主要工作

本课题以川崎 FS003 手工业机器人（机械手）为研究对象，围绕工业机器人的结构设计、运动学分析、强度校核及仿真控制等核心内容展开系统性研究，综合运用机械原理、机械设计、计算机辅助设计、ANSYS 有限元分析、MATLAB 编程、机器人技术及半实物仿真等多学科知识与工具，完成机电一体化的综合训练。课题组按照课程设计要求，对川崎 FS003 机械手进行了从理论分析、设计建模到仿真验证的全流程研究，具体主要工作内容如下：

第一节 机械手结构建模

运用三维设计软 SolidWorks，按照川崎 FS003 机械手的结构参数与设计要求，完成各零件的三维造型设计，精准还原零件的尺寸、特征及装配关系；通过软件的装配功能进行虚拟装配，检查零部件之间的干涉情况，优化装配结构，生成机械手的整体三维装配模型，为后续有限元分析与运动仿真提供实体模型支撑。

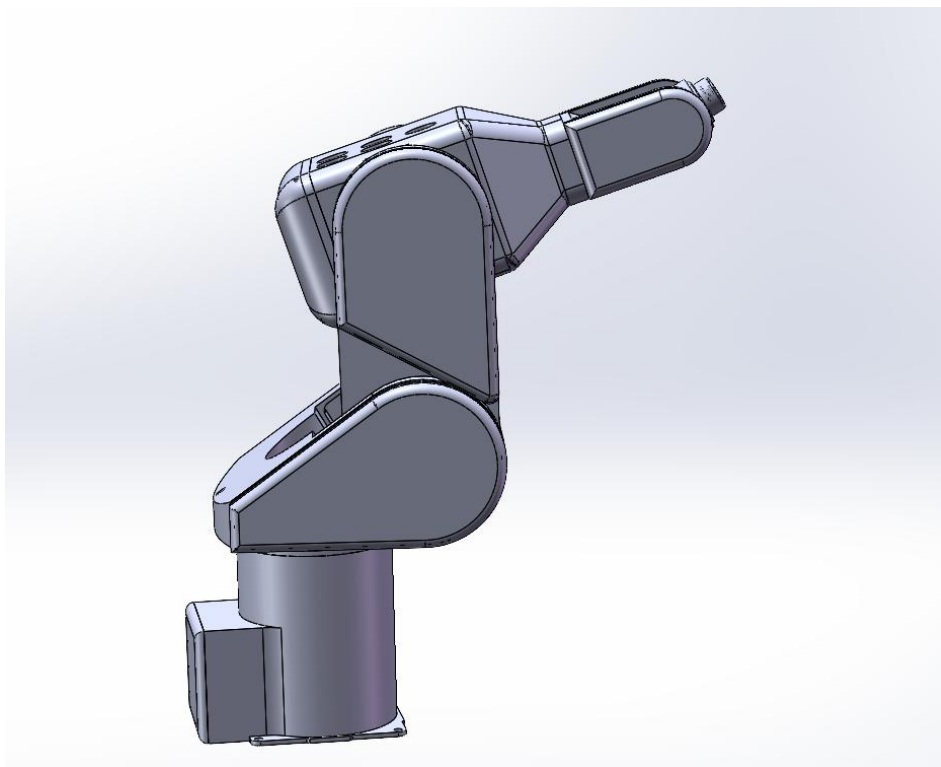


图 1.1 FS003 机械手装配图

第二节 机械手运动学分析与仿真

利用 MATLAB 对川崎 FS003 机械手的工作空间进行分析，通过数值计算与图形化展示，确定机械手末端执行器的可达工作范围，验证其作业空间是否满足工业应用需求。

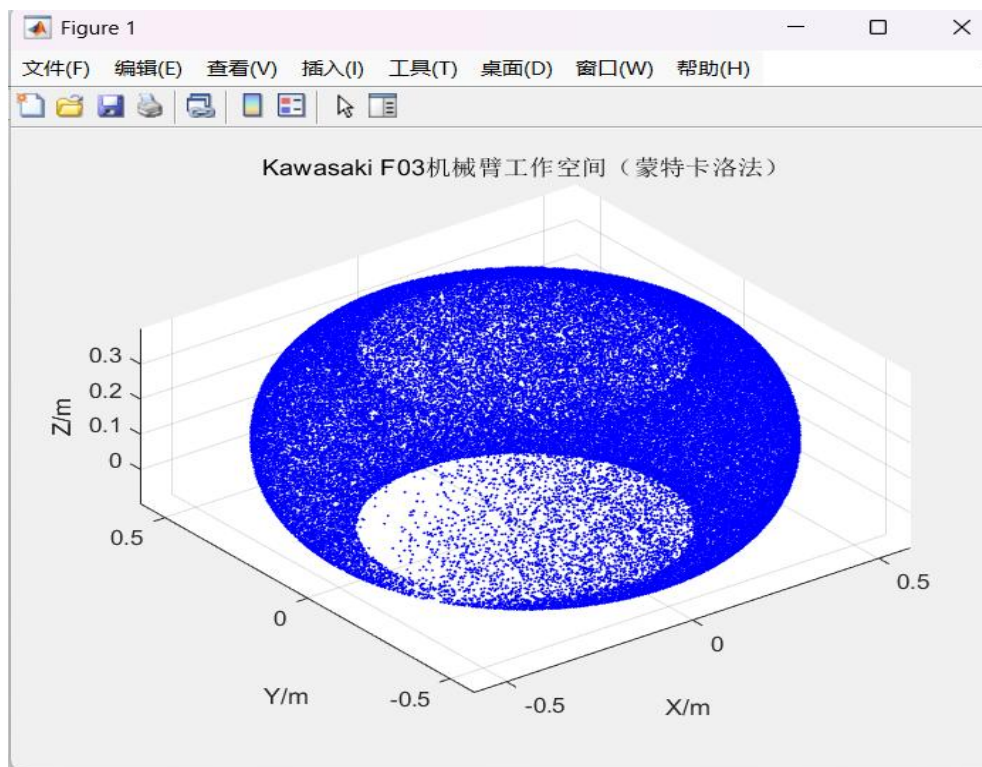


图 1.2 FS003 机械手运动学分析

第三节 机械手结构强度校核与有限元分析

利用 ANSYS 进行动力学分析（如模态分析、瞬态动力学分析），求解机械手的固有频率与振型，避免共振现象；分析动态载荷下零件的应力变化规律，评估机械手在动态工作状态下的结构稳定性与可靠性。

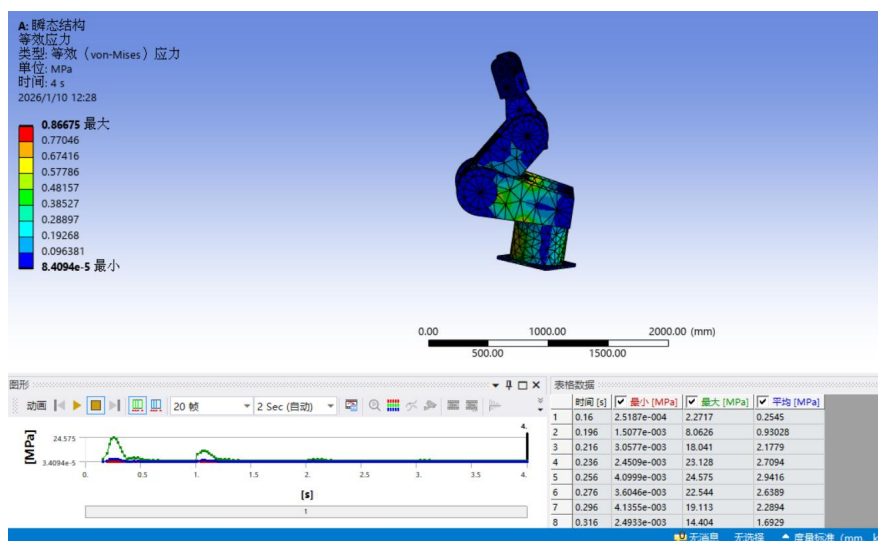


图 1.3 机械手整机应力云图

第四节 Matlab 工具箱函数仿真

基于关节空间与笛卡尔空间，设计直线、圆弧、多项式等轨迹规划算法，编写 MATLAB 程序实现机械手的轨迹规划与运动仿真，输出关节角、速度、加速度随时间的变化曲线，分析轨迹的平滑性与运动特性。

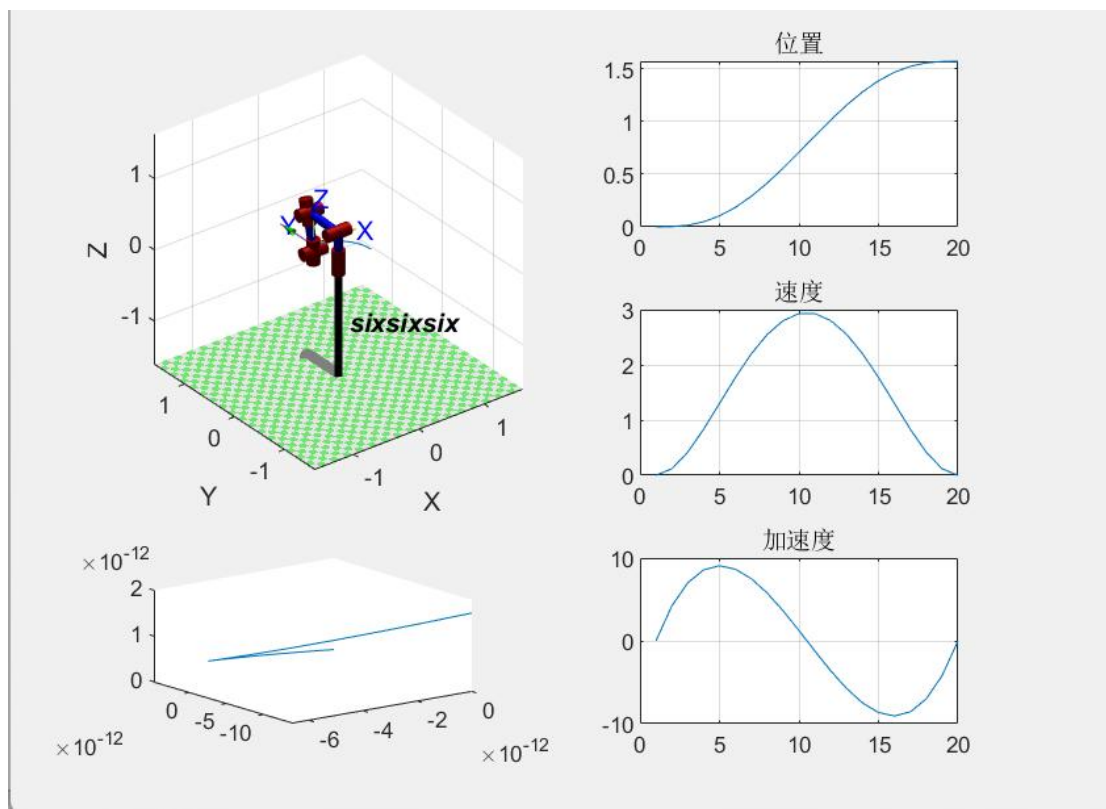


图 1.4 matlab 工具箱函数仿真

第二章 Simulink 子课题主要工作

第一节 设计基础

1.1 Simulink 设计基础

Simulink 仿真建模：利用机器人半实物平台，基于 Simulink 搭建川崎 FS003 机械手的控制系统模型，整合运动学算法、轨迹规划模块、驱动控制模块及传感器反馈模块，构建闭环仿真系统，模拟机械手的运动控制过程，验证控制算法的正确性与响应特性。

半实物轨迹控制实验：将 Simulink 仿真模型与机器人半实物平台的硬件（如伺服驱动器、电机、传感器、控制器）进行联合调试，实现对川崎 FS003 机械手的半实物轨迹控制；通过实验采集机械手的实际运动轨迹数据，与仿真轨迹对比分析，修正控制参数，优化控制算法，提升机械手的轨迹跟踪精度与运动控制性能。

第二节 确定 DH 参数

采用改进法建立六轴机械臂坐标系，建立的坐标系如下图 2.1：

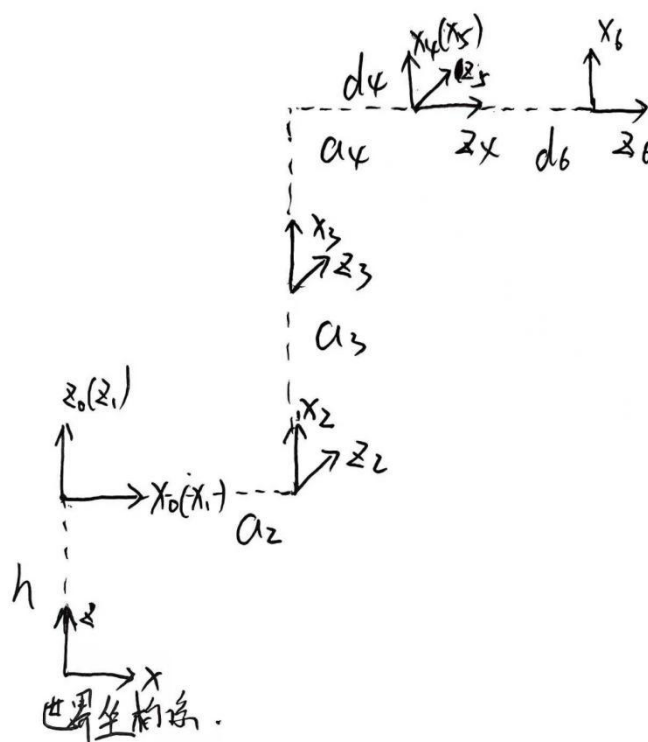


图 2.1 改进法坐标系

通过改进法算出 D-H 参数表：

	alpha	a	theta_offset	d
1	0	0	0	0
2	-pi/2	a2	-pi/2	0
3	0	a3	0	0
4	-pi/2	a4	0	d4
5	pi/2	0	0	0
6	-pi/2	0	0	d6

a2=0.02315m

a3=0.26m

a4=0.0276m

d4=0.2745m

d6=0.150m

第三节 正运动学推导及 simulink 设计

机器人正运动学所研究的内容是：给定机器人各关节的角度或位移，求解计算机器人末端执行器相对于参考坐标系的位置和姿态。

改进法坐标系 i-1 到坐标系 i 的齐次变换矩阵 A_i 公式如下：

$$A_i = Rot(X, \alpha) * Trans(X, a) * Rot(Z, \theta) * Trans(Z, d) \quad (3.1)$$

式（3.1）由于是相对动坐标系进行运算，所以算子右乘

$$Rot(X, \alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.2)$$

$$Trans(X, a) = \begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.3)$$

$$Rot(Z, \theta) = \begin{bmatrix} \cos(\theta_i + \theta_{offset_i}) & -\sin(\theta_i + \theta_{offset_i}) & 0 & 0 \\ \sin(\theta_i + \theta_{offset_i}) & \cos(\theta_i + \theta_{offset_i}) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.4)$$

$$Trans(Z, d) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.5)$$

为方便显示 $\cos(\theta_i + \theta_{offset})$ 用 c_t, 表示 $\sin(\theta_i + \theta_{offset})$ 用 s_t

表示, $\cos\alpha_i$ 用 c_a 表示, $\sin\alpha_i$ 用 s_a 表示, 最后算出的 A_i 为:

$$A_i = \begin{bmatrix} c_t & -s_t & 0 & a \\ s_t * c_a & c_t * c_a & -s_a & -d * s_a \\ s_t * s_a & c_t * s_a & c_a & d * c_a \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.6)$$

机械臂末端执行器相对于机身坐标系的齐次变换矩阵为:

$${}^0_6T = A_1 * A_2 * A_3 * A_4 * A_5 * A_6 \quad (3.7)$$

当给定 $\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6$, 通过式 (3.7) 会得到末端执行器相对于机身坐标系 (也可以是世界坐标系) 的齐次变换矩阵:

$${}^0_6T = \begin{bmatrix} nx & ox & ax & px \\ ny & oy & ay & py \\ nz & oz & az & pz \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.8)$$

很明显末端执行器在机身坐标系下的坐标为 $x=px, y=py, z=pz$, 但是末端执行器在机身坐标系下的位姿 $ropy$ 是隐藏在左上角的 3×3 的旋转矩阵中的, 为了知晓末端执行器在机身坐标系下的位姿需要通过左上角的 3×3 的旋转矩阵和欧拉角旋转矩阵 $R = Rx(roll) * Ry(pitch) * Rz(yaw)$ 联立求解算出。

以下为计算过程:

$$Rx(roll) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(roll) & -\sin(roll) \\ 0 & \sin(roll) & \cos(roll) \end{bmatrix}$$

$$Ry(pitch) = \begin{bmatrix} \cos(pitch) & 0 & \sin(pitch) \\ 0 & 1 & 0 \\ -\sin(pitch) & 0 & \cos(pitch) \end{bmatrix}$$

$$Rz(yaw) = \begin{bmatrix} \cos(yaw) & -\sin(yaw) & 0 \\ \sin(yaw) & \cos(yaw) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3.9)$$

$$R = \begin{bmatrix} \cos(pitch) * \cos(yaw) & -\cos(pitch) * \sin(yaw) & \sin(pitch) \\ \cos(roll) * \sin(yaw) + \cos(yaw) * \sin(pitch) * \sin(roll) & \cos(roll) * \cos(yaw) - \sin(pitch) * \sin(roll) * \sin(yaw) & -\cos(pitch) * \sin(roll) \\ \sin(roll) * \sin(yaw) - \cos(roll) * \cos(yaw) * \sin(pitch) & \cos(yaw) * \sin(roll) + \cos(roll) * \sin(pitch) * \sin(yaw) & \cos(pitch) * \cos(roll) \end{bmatrix} \quad (3.10)$$

第一种情况, 如果 $R(2,3) = R(3,3) = 0, R(1,3) \neq 0, R(1,3) = \pm 1$ 此时为奇异点, $pitch$ 为 $\pm\pi/2$,

① 带入 $pitch=\pi/2$ 于矩阵 R 中, 得:

$$R = \begin{bmatrix} 0 & 0 & 1 \\ \sin(\text{roll} + \text{yaw}) & \cos(\text{roll} + \text{yaw}) & 0 \\ -\cos(\text{roll} + \text{yaw}) & \sin(\text{roll} + \text{yaw}) & 0 \end{bmatrix} \quad (3.11)$$

此时只能算出 $\text{roll} + \text{yaw} = \text{atan2}(R(2,1), R(2,2))$ ，所以选 roll 等于上一次计算的 roll ， $\text{yaw} = \text{atan2}(R(2,1), R(2,2)) - \text{last_roll}$ 。

② 带入 $\text{pitch} = -\pi/2$ 于矩阵 R 中，得：

$$R = \begin{bmatrix} 0 & 0 & -1 \\ \sin(\text{roll} - \text{yaw}) & \cos(\text{roll} - \text{yaw}) & 0 \\ \cos(\text{roll} - \text{yaw}) & -\sin(\text{roll} - \text{yaw}) & 0 \end{bmatrix} \quad (3.12)$$

此时只能算出 $\text{roll} - \text{yaw} = \text{atan2}(R(2,1), R(2,2))$ ，同样选 roll 等于上一次计算的 roll ， $\text{yaw} = -\text{atan2}(R(2,1), R(2,2)) + \text{last_roll}$ 。

第二种情况，非奇异点， $\text{pitch} \neq \pm \pi/2$

$$\text{roll} = \text{atan2}(-R(2,3), R(3,3)) \quad (3.13)$$

$$\text{pitch} = \text{atan2}(R(1,3), \cos(\text{roll}) * R(3,3) - \sin(\text{roll}) * R(2,3)) \quad (3.14)$$

如果 $\cos(\text{pitch}) > 0$ ， $\text{yaw} = \text{atan2}(-R(1,2), R(1,1))$

如果 $\cos(\text{pitch}) < 0$ ， $\text{yaw} = \text{atan2}(R(1,2), -R(1,1))$

为了验证其正确性，需对 simulink 进行设计，首先找到 $\text{user-defined function}$ 中的 MATLAB Function 模块，拖出，双击后开始编写 matlab 代码，这里设计了两个 MATLAB Function 模块，一个是正运动学的函数，另一个是齐次变换矩阵计算位姿的函数，如下图 3.1 所示。

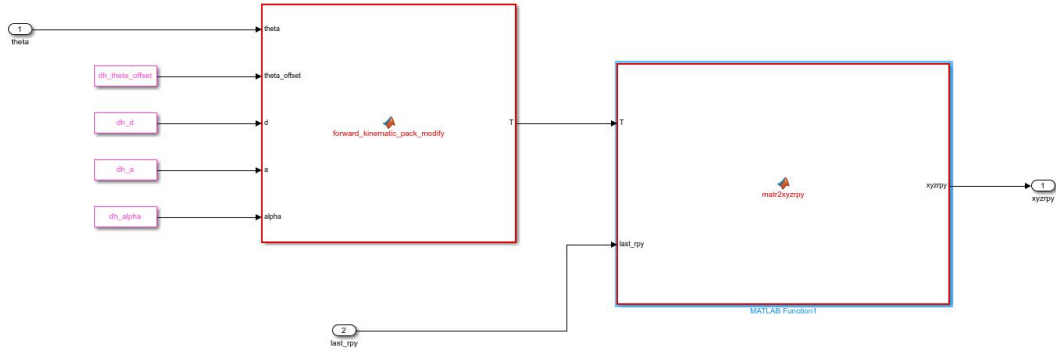


图 3.1 正解 simulink 设计图

代码详见附件文件夹。

第四节 逆运动学推导及 simulink 设计

机器人逆运动学研究的是在末端执行器的位姿为已知的情况下，如何求解各个关节的角度。这是由实际工作需要决定的，首先需要工具达到某个位姿，由此反求出关节角度，再由角度计算出控制器输出控制电机的信号，从而使工具到达预期的位姿。

机器人的运动学反解存在的区域称为机器人的工作空间。工作空间是操作臂的末端能够到达的空间范围，即末端能够到达的目标点集合。值得指出的是，工作空间应该严格地区分为两类：

灵活（工作）空间指机器人末端能够以任意方位到达的目标点集合。

可达（工作）空间指机器人末端至少在一个方位上能够到达的目标点集合。

机器人操作臂运动学逆解的数目决定于关节数目、连杆参数（对于旋转关节操作臂指的是 d_i 、 a_i 和 α_i ）和关节角度范围。在解运动学方程时，碰到的另一问题是逆解不唯一（存在多解）。在工作空间中任何点，机械手能以多种姿势到达。

在求解逆解时，需要给定末端的坐标和位姿 $[x, y, z, \text{roll}, \text{pitch}, \text{yaw}]$ ，将其转换为 4X4 的齐次变换矩阵 T_{target} 作为逆解程序的输入。转换过程如下：

$$T_{target}(1,4) = x$$

$$T_{target}(2,4) = y$$

$$T_{target}(3,4) = z$$

$$T_{target}(1:3,1:3) = R_x(\text{roll}) * R_y(\text{pitch}) * R_z(\text{yaw}) \text{ 详见式 (3.9)}$$

$$T_{target}(4,:) = [0,0,0,1]$$

得到 T_{target} 后，做出等式：

$$T_{target} = {}^0T \quad (4.1)$$

详见式（3.7），以下求解 T_{target} 简化用式（3.8）表示。

接下来开始求逆解，12 个等式，6 个未知数，首先在腕关节处建一个腕部坐标系 $\{W\}$ ，它的姿态与坐标系 $\{6\}$ 相同，只是原点沿 Z_6 轴平移了 $-d_6$ ，移到了三轴交点处，从而得到：

$${}^6T_w = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -d_6 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4.2)$$

将式（4.2）分别右乘式（4.1）的左右两边得到：

$$T_w = {}^0T \quad (4.3)$$

$$T_w = \begin{bmatrix} nx & ox & ax & px - ax * d_6 \\ ny & oy & ay & py - ay * d_6 \\ nz & oz & az & pz - az * d_6 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4.4)$$

4.1 计算 theta1

通过 $T_w(2,4) = {}^0T(2,4) = 0$ ，则 $\cos\theta_1 * (py - ay * d_6) - \sin\theta_1 * (px - ax * d_6) = 0$ ，设 $p_{wy} = py - ay * d_6$ ， $p_{wx} = px - ax * d_6$ 。

$$\theta_1 = \text{atan2}(\pm p_{wy}, \pm p_{wx}) \quad (4.5)$$

4.2 计算 theta3

设 $K_1 = T_w(1,4) - a_2$ ， $K_2 = T_w(3,4)$

$$K_1^2 + K_2^2 = ({}^0T(1,4) - a_2)^2 + {}^0T(3,4)^2$$

$$K_1^2 + K_2^2 = a_3^2 + a_4^2 + d_4^2 + 2 * a_3 * (a_4 * \cos\theta_3 - d_4 * \sin\theta_3)$$

$$\text{设 } K = \frac{K_1^2 + K_2^2 - (a_3^2 + a_4^2 + d_4^2)}{2 * a_3} = \sqrt{a_4^2 + d_4^2} * \left(\frac{a_4}{\sqrt{a_4^2 + d_4^2}} * \cos\theta_3 - \frac{d_4}{\sqrt{a_4^2 + d_4^2}} * \sin\theta_3 \right)$$

$$\text{设 } \sin\phi_1 = \frac{a_4}{\sqrt{a_4^2 + d_4^2}}, \cos\phi_1 = \frac{d_4}{\sqrt{a_4^2 + d_4^2}}$$

$$\text{则 } \sin(\phi_1 - \theta_3) = \frac{K}{\sqrt{a_4^2 + d_4^2}}, \cos(\phi_1 - \theta_3) = \pm \frac{\sqrt{a_4^2 + d_4^2 - K^2}}{\sqrt{a_4^2 + d_4^2}}$$

最后算出:

$$\theta_3 = \text{atan2}(a_4, d_4) - \text{atan2}(K, \pm \sqrt{a_4^2 + d_4^2 - K^2}) \quad (4.6)$$

4.3 计算 theta2

将 θ_3 带入 ${}^0T(1,4)$ 和 ${}^0T(3,4)$ 联立 K_1 和 K_2 即可求出 θ_2 。

$$\theta_2 = \text{atan2}(m_2 * K_1 - m_1 * K_2, m_1 * K_1 + m_2 * K_2) \quad (4.7)$$

$$m_1 = a_4 * \cos\theta_3 - d_4 * \sin\theta_3 + a_3$$

$$m_2 = -a_4 * \sin\theta_3 - d_4 * \cos\theta_3$$

4.4 计算 theta5

由于已求出关节 1、关节 2 和关节 3 的角度可以将它们的齐次变换矩阵转移到矩阵的左边，如下：

$$\text{inv}(A_3) * \text{inv}(A_2) * \text{inv}(A_1) * T_w = A_4 * A_5 * A_6 * {}^6T_w$$

$$\text{此处简化 } L = \text{inv}(A_3) * \text{inv}(A_2) * \text{inv}(A_1) * T_w, R = A_4 * A_5 * A_6 * {}^6T_w。$$

$$R(2,3) = \cos\theta_5 = L(2,3)$$

$$\theta_5 = \pm \arccos(L(2,3)) \quad (4.7)$$

4.5 计算 theta4 和 theta6

① $\theta_5 < 0$ 时：

$$\theta_4 = \text{atan2}(-L(3,3), L(1,3))$$

$$\theta_6 = \text{atan2}(L(2,2), -L(2,1))$$

② $\theta_5 > 0$ 时：

$$\theta_4 = \text{atan2}(L(3,3), -L(1,3))$$

$$\theta_6 = \text{atan2}(-L(2,2), L(2,1))$$

③ $\theta_5 = 0$ 时：此时关节 4 和关节 6 转轴是共线的， θ_4 可以任意取值， θ_6 根据 θ_4 的值而定。

$$\theta_4 + \theta_6 = \text{atan2}(-L(1,2), L(1,1))$$

θ_4 选上一次计算的结果， θ_6 就可以算出来了。

4.6 simulink 设计

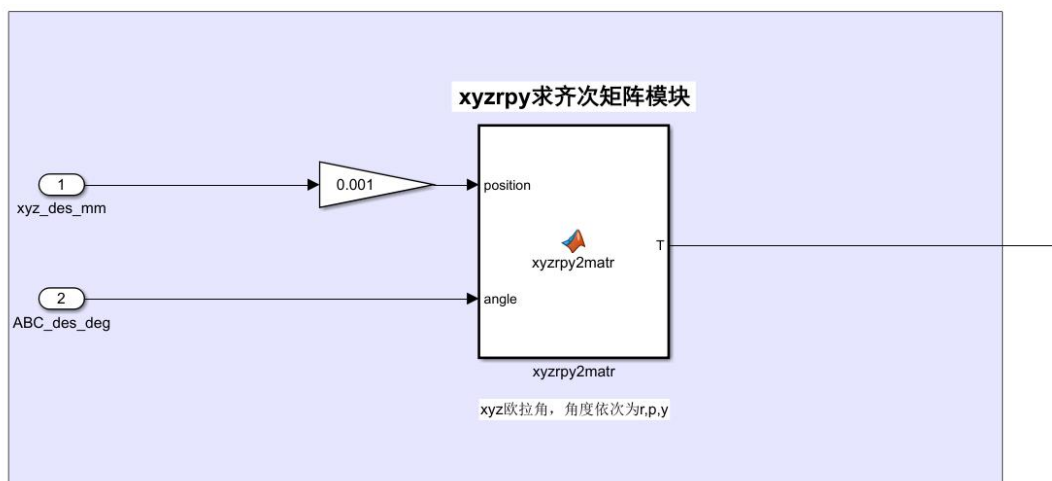


图 4.1 xyzrpy 求齐次矩阵模块

首先创建两个 inport (xyz 和 rpy) 作为该模块的输入, xyz 输入的数据单位为 mm 需要通过一个数值为 0.001 的 Gain 将 mm 单位转换为 m, 然后通过 xyzrpy2matr 转换为 4x4 的齐次矩阵作为逆解的输入。

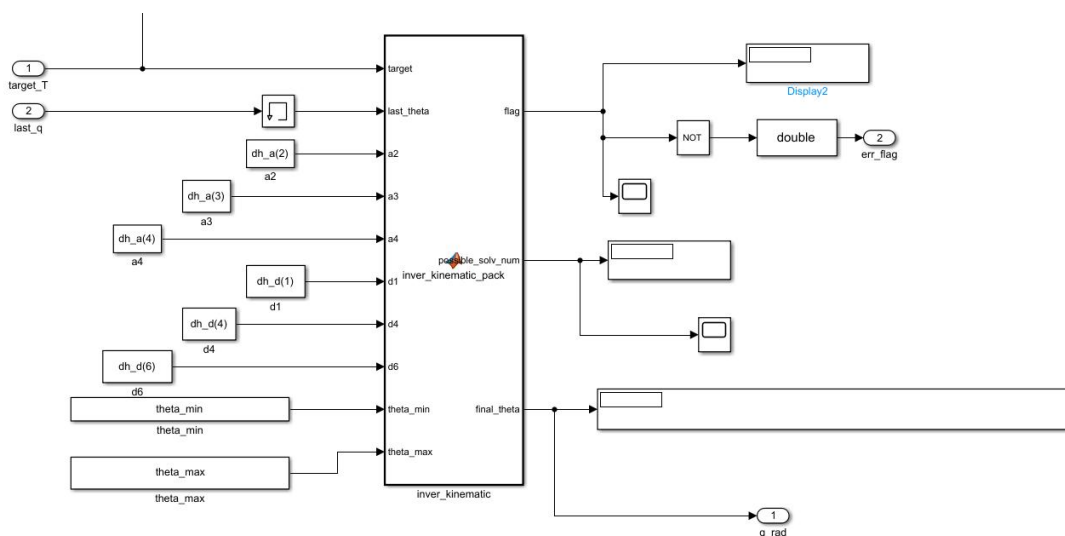


图 4.2 逆运动学模块

a_2 , a_3 , a_4 , d_4 , d_6 , $\theta_{\min}$, $\theta_{\max}$ 均是已知的所以设置为 Constant 数据, last_theta 由于是上一次计算得出的角度, 所以需要加一个 memory 模块, flag 为判断是否有正确解的标志, $\text{flag}=1$ 逆解有解, $\text{flag}=0$ 逆解无解, possible_solv_num 为逆解有解的组数, 见上述推导最多有八组解 (θ_1 两个, θ_3 两个, θ_5 两个, $2 \times 2 \times 2 = 8$), final_theta 为最优的一组解。本函数是通过

- ①结构限制: θ 应该在 $\theta_{\min}$ 和 $\theta_{\max}$ 之间
- ②位置变动最小: 让关节运动量最小。

第五节 关节位置梯型加减速及 simulink 设计

在通过逆解得到每个关节的目标角度后，需对关节运动进行规划，这里是在关节空间进行梯型速度规划。

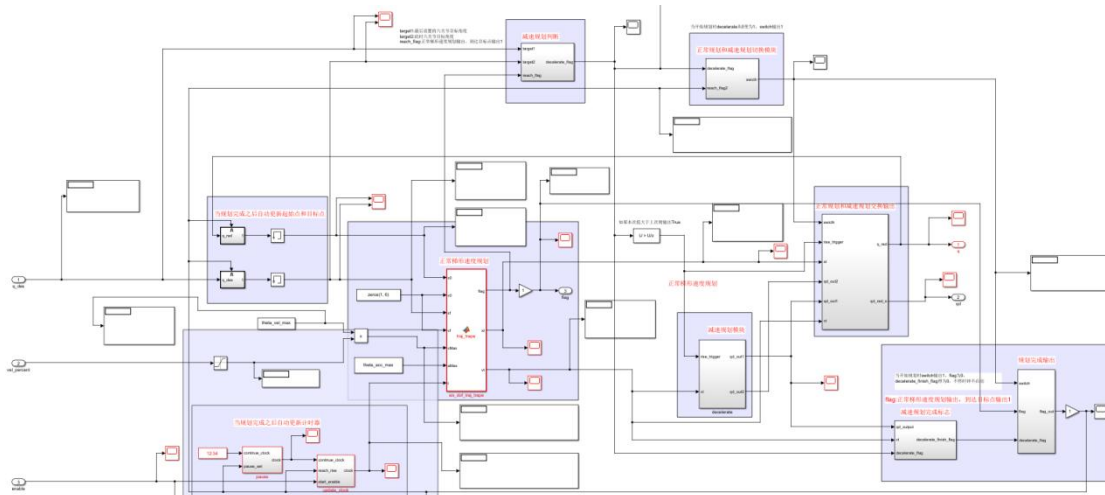


图 5.1 关节空间规划模块

上图为关节空间规划模块的概览图，里面包含了核心的六自由度梯型加减速模块，计时器自动更新模块，减速规划模块以及正常规划与减速规划切换模块。

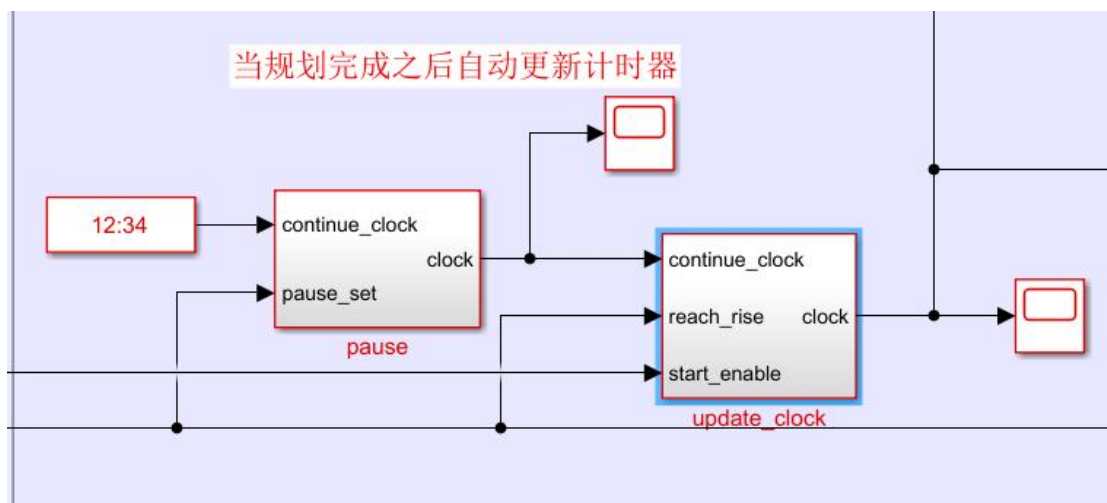


图 5.2 计时器自动更新模块

数字时钟模块 Digital Clock 在模型运行时就开始计时，而每段轨迹规划计时器都要从零开始，所以当一段轨迹规划完成之后，flag 会由 0 变为 1，此时 pause 子系统负责暂停时钟，update 子系统负责让时钟归零。

如果我们在运动过程中修改目标关节角度，为了马上重新规划新的轨迹，则我们需要先让关节先减速到零，然后再重新规划，这就涉及轨迹切换的。首先我们判断是否需要减速，如下图所示：

target1:最后设置的六关节目标角度
 target2:此时六关节目标角度
 reach_flag:正常梯形速度规划输出，到达目标点输出1



图 5.3 减速规划判断模块

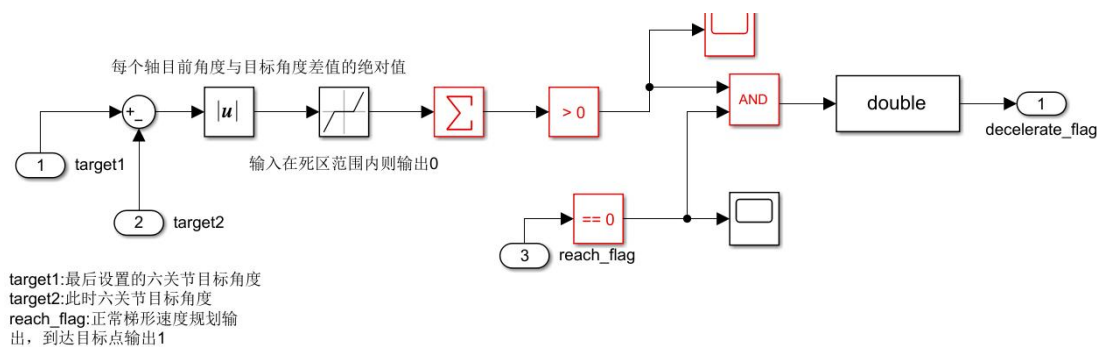


图 5.4 减速规划判断模块内部

如果上一时刻的目标角度和新的期望目标角度不一样，并且上一段轨迹没有规划完成(flag=0)，则表示需要进行减速，decelerate_flag = 1，否则 decelerate_flag = 0。另外，正常规划和减速规划需要进行切换，以此从两种规划得到结果选择其中一种进行输出，切换模块如下图所示：

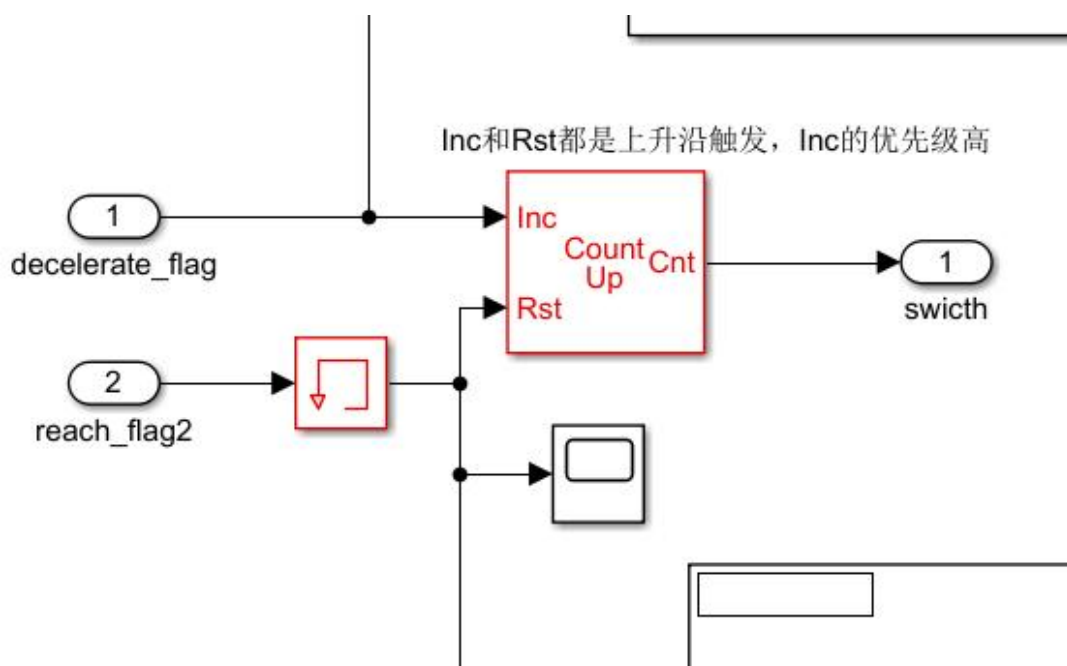


图 5.5 正常规划和减速规划切换模块

decelerate_flag 输入表示是否需要进行减速规划，当它由零变为 1 时，会产生一个上升沿，触发计数器等于 1，表示需要进行减速规划。等减速完成之后 Reach_flag2 会等于 1，此时计数器会归零，表示接下来需要进行正常的梯形速度规划。

减速规划模块的原理是让关节按照某一减速度值进行减速，如下图所示：

decelerate_flag 从 0 变为 1，rise_trigger 在一个 SAMPLE_TIME 内为 1

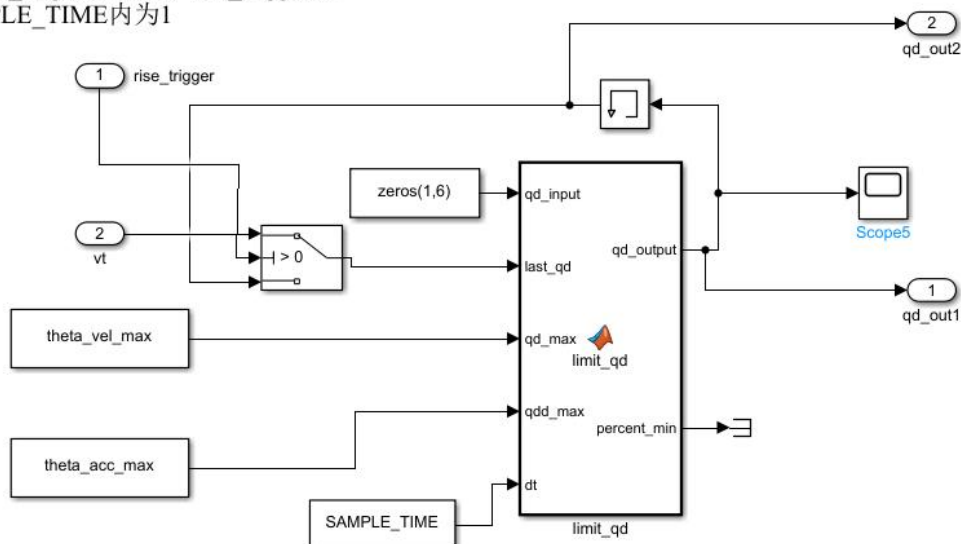


图 5.6 减速规划模块

当检测 rise_trigger 为 1 时，表示需要进行减速规划，此时需要把正常轨迹规划的速度 vt 作为模块 last_qd 输入源，这样就可以输入减速前的初始速度了。等到后面周期 rise_trigger 为 0 时，只需要 qd_output 作为输入即可。

当减速规划完成后也需要一个完成标志 decelerate_finish_flag，如下图：

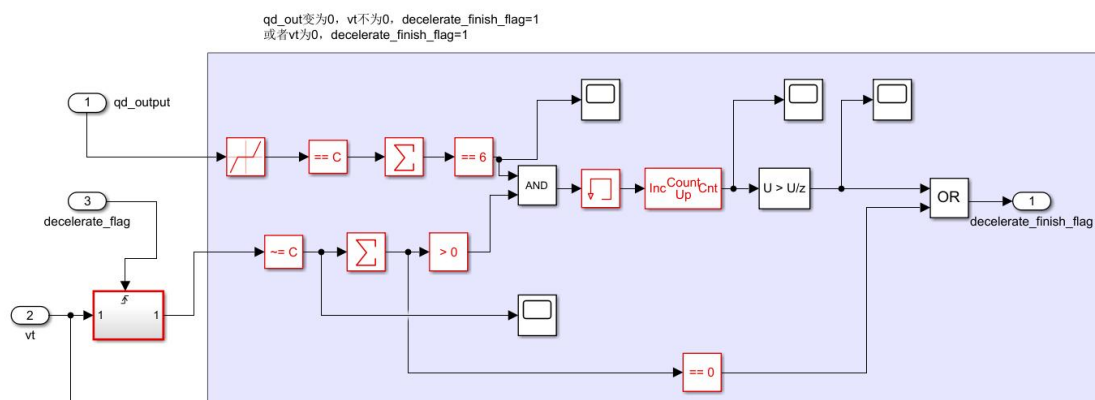


图 5.7 减速规划完成标志模块

因为最终输出的关节角度和角速度都是一路信号，所以我们要有一个模块在恰当的时候切换输出，如下图所示：

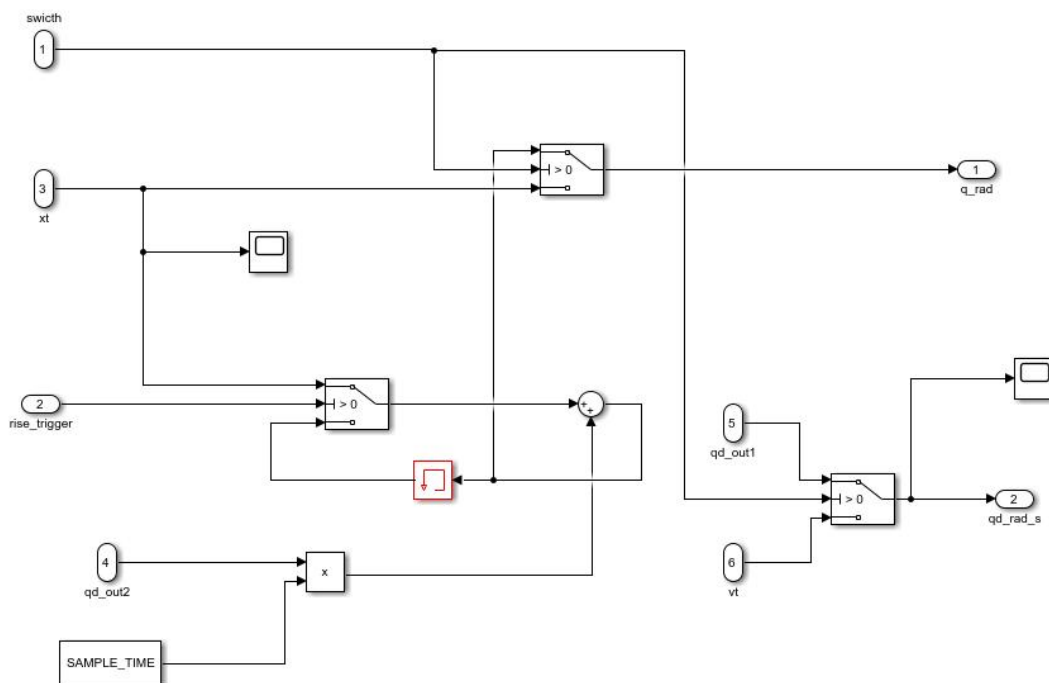


图 5.8 正常规划和减速规划切换模块

Switch 是切换信号，当它等于 1 时切换到减速规划，当它等于 0 时切换到梯形速度规划。**rise_trigger** 是减速规划的触发脉冲信号，用来触发保存梯形速度规划最后一个周期的关节角度值。**xt** 是梯形速度规划的角度值。**vt** 是梯形速度规划的角速度值。**qd_out1** 和 **qd_out2** 是减速规划的速度值，它和仿真周期相乘就可以得到角度步长。角度步长加上上一个周期的角度值即可得到新周期的角度值。

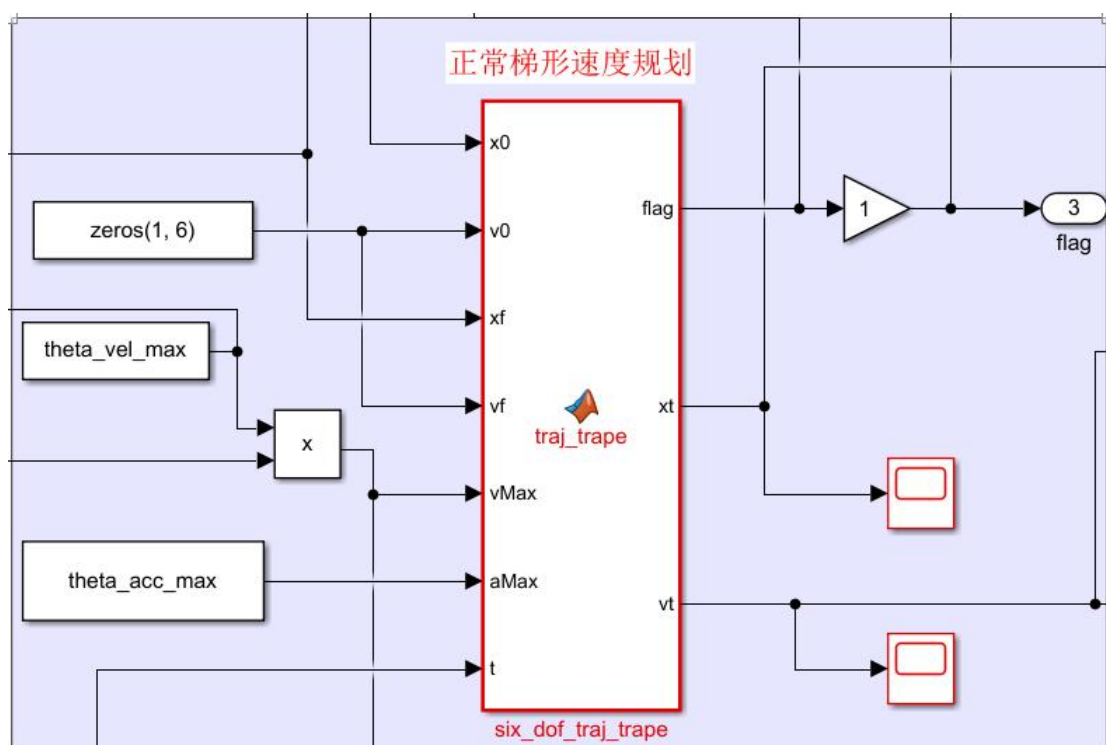


图 5.9 关节空间梯型加速减速模块

梯型加速减速顾名思义就是速度从零加到 V_{max} 的距离加上速度从 V_{max} 减到零的距离如果小于你要走的距离就加上 $V=V_{max}$ 的匀速阶段，大于的话就去除 $V=V_{max}$ 的匀速阶段。

① $\theta_f - \theta_0 >$ 加速减速完成走到距离时，开始的加速段 $0 < t < t_1$ ，角度时间曲线为抛物线形： $\theta = a_0 + b_0 + c_0 * t^2$ ；中间的恒速运动段 $t_1 < t < t_2$ ，角度时间曲线为直线形： $\theta = a_1 + b_1 * t$ ；最后的减速段 $t_2 < t < t_f$ ，角度时间曲线也是抛物线形 $\theta = a_2 + b_2 + c_2 * t^2$ 。

$$\begin{aligned} a_0 &= \theta_0 \\ b_0 &= \dot{\theta}_0 \\ c_0 &= \ddot{\theta}_{max}/2 \\ c_2 &= -\ddot{\theta}_{max}/2 \\ b_1 &= \dot{\theta}_{max} \end{aligned}$$

联立即可求得： a_1 、 a_2 、 b_2 、 t_1 和 t_2

② $\theta_f - \theta_0 >$ 加速减速完成走到距离时，就是单调的加速减速。对于相同的目标速度 $\dot{\theta}_f$ ，找到中间的临界值速度。

加速段位置： $x_1(t) = x_0 + v_0 t + \frac{1}{2} a_1 t^2$ (其中 $a_1 = n \cdot a_{Max}$)

加速段速度： $v_1(t) = v_0 + a_1 t$

减速段位置： $x_2(t) = x_2 + v_1(t_2) \cdot (t - t_2) - \frac{1}{2} a_2 (t - t_2)^2$ (其中 $a_2 = n \cdot a_{Max}$)

减速段速度： $v_2(t) = v_1(t_2) - a_2 (t - t_2)$

$$x_1(t_2) = x_2(t_2) \quad x_0 + v_0 t_2 + \frac{1}{2} a_1 t_2^2 = x_2 + v_1(t_2) \cdot 0 - \frac{1}{2} a_2 \cdot 0^2 \quad (5.1)$$

$$v_1(t_2) = v_2(t_2) \quad v_0 + a_1 t_2 = v_1(t_2) \quad (5.2)$$

$$x_2(t_f) = x_f \quad x_2 + v_1(t_2) \cdot (t_f - t_2) - \frac{1}{2}a_2(t_f - t_2)^2 = x_f \quad (5.3)$$

$$v_2(t_f) = v_f \quad v_1(t_2) - a_2(t_f - t_2) = v_f \quad (5.4)$$

上述四式整合得 $2a^2t_2^2 + 4av_0t_2 + [2a(x_0 - x_f) + v_0^2 - v_f^2] = 0$

$$At_2^2 + Bt_2 + C = 0$$

$$A = 2(2c_0)^2 = 8c_0^2$$

$$B = 4(2c_0)v_0 = 8c_0v_0 = 8c_0 \cdot b_0 \text{ (因为 } b_0 = v_0 \text{)}$$

$$C = 2(2c_0)(x_0 - x_f) + v_0^2 - v_f^2 = 4c_0(x_0 - x_f) + b_0^2 - v_f^2$$

从标准解公式 $t_2 = [-B \pm \sqrt{B^2 - 4AC}]/(2A)$

临界值速度 $m = \sqrt{B^2 - 4AC}$ ，然后可求得： a_1 、 a_2 、 b_2 、 t_1 和 t_2

第六节 simulink 导入机械臂模型

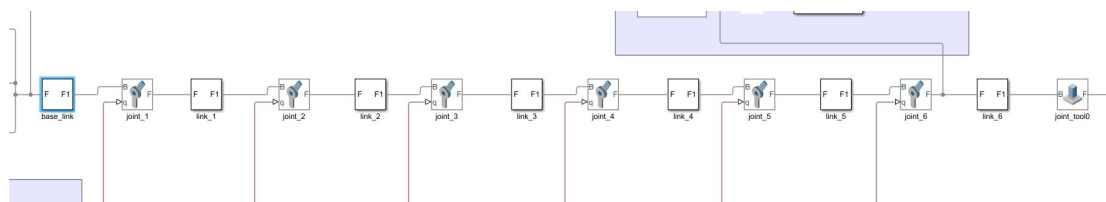


图 6.1 机械臂模型组合图

图 6.1 中的 base_link、link_1、link_2、link_3、link_4、link_5 和 link_6 等 block 中均可导入模型，分别对应的是关节 1、关节 2、关节 3、关节 4、关节 5、关节 6 以及末端执行器，以 base_link 为例，如下图：

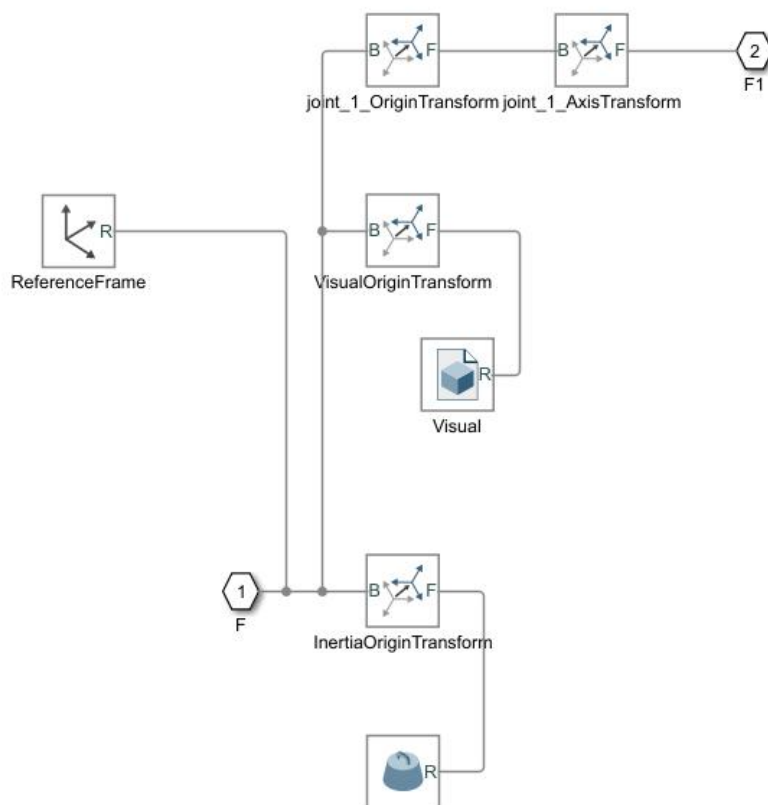


图 6.2 base_link 模块

图 6.2 中 ReferenceFrame 为模型构建时的原点坐标，visual 为模型可视化配置见下图：

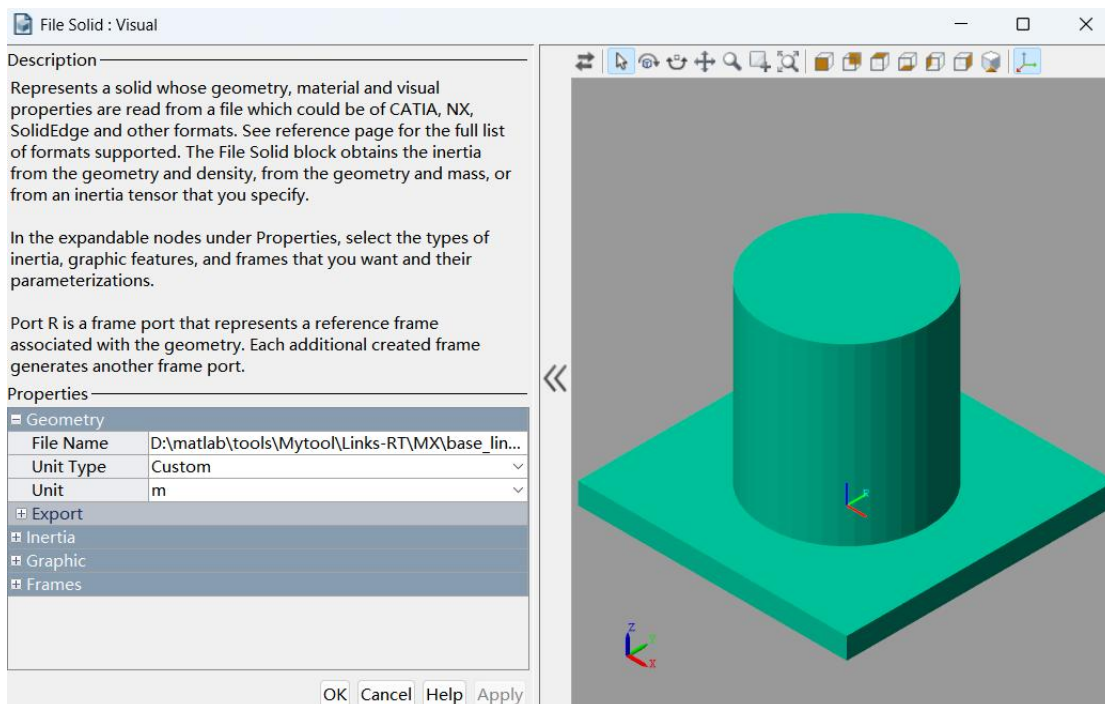


图 6.3 visual

File Name 为 STL 模型所在路径，Unit 为模型单位，这里时 m。

图 6.2 中的 inertia 为模型的惯性属性，剩下的 Rigid Transform 为坐标变换模块，见下图：

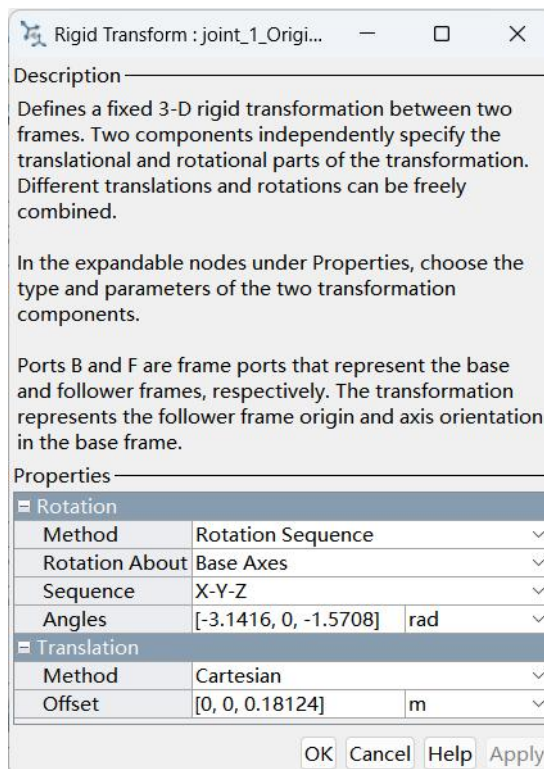


图 6.4 Rigid Transform

Rotation: 坐标系旋转，**Sequence** 为旋转的顺序，**Angles** 为旋转的角度（弧度制），图 6.4 中是先绕 X 轴旋转 $-\pi$ ，再绕 Y 轴旋转 0，最后绕 Z 轴旋转 $-\pi/2$ ，旋转方向依据右手定则。

Translation: 坐标系平移，图 6.4 中是先向 X 轴平移 0m，再向 Y 轴平移 0m，最后向 Z 轴平移 0.18124m。

图 6.2 中的 **visual** 为模型观察时的位置和姿态可通过 **VisualOriginTransform** 进行调整到合适角度，见图 6.1，F1 坐标系是参与到关节转动的，如 **base_link** 的 F1 是关节 1 的旋转坐标系是 **joint_1** 的输入，所以 F1 的 Z 轴朝向应与关节旋转轴平行，**base_link** 的 F1 是 **ReferenceFrame** 通过 **joint_1_OriginTransform** 和 **joint_1_AxisTransform** 变换过来的，如果 F1 不正确可以通过调整 **joint_1_OriginTransform** 和 **joint_1_AxisTransform** 来校正。

第七节 simulink 轨迹模块设计

上述过程是单点的运行过程，如果需要多点运行，需要加一个 MATLAB Function 来作为给点函数，如下图：

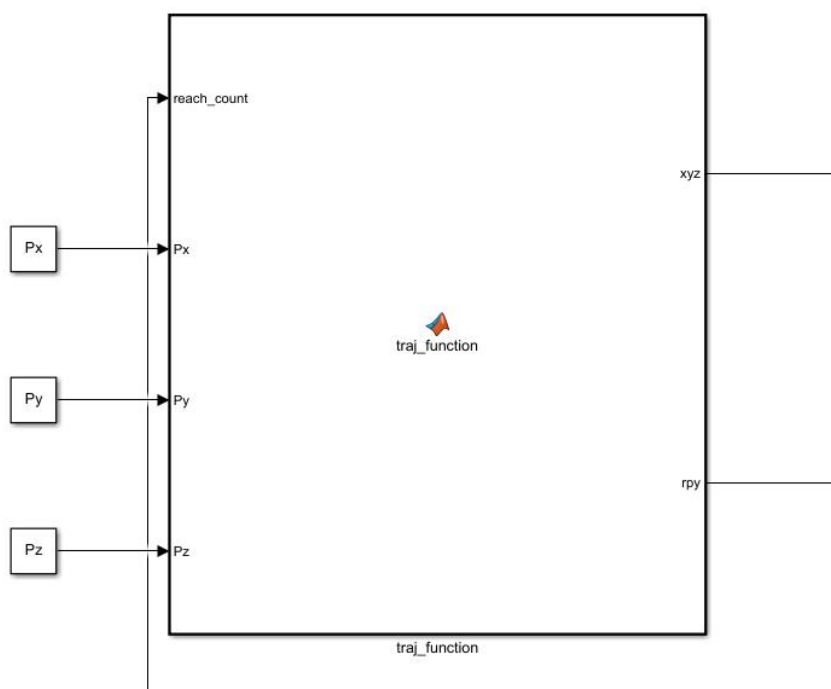


图 7.1 traj_function 模块

P_x , P_y , P_z 为提前算好的坐标点集合，**reach_count** 为已经到达点的个数。程序如下图所示：

```

function [xyz, rpy] = traj_function(reach_count, Px, Py, Pz)
%base_xyz = [297.7, 0, 385.1];
base_xyz = [274.5, 0, 405.1+0.4];%0.4: 第一个SAMPLE_TIME
base_rpy = [180, 0, 0];
delta_xyz = [0, 0, 0];
delta_rpy = [0, 0, 0];

%   delta_xyz(1) = Px(reach_count+1);
%   delta_xyz(2) = Py(reach_count+1);
%   delta_xyz(3) = Pz(reach_count+1);

if reach_count == 1
    delta_xyz = [100, 100, 0];
    delta_rpy = [0, 0, 0];
elseif reach_count == 2 || reach_count == 3
    delta_xyz = [-100, -100, 0];
    delta_rpy = [0, 0, 0];
end

xyz = base_xyz + delta_xyz;
rpy = base_rpy + delta_rpy;
end
    
```

图 7.2 traj_function 模块内部程序

当到达初始点时改变 Δ_{xyz} 和 Δ_{rpy} ，如果需要许多个点可以将中间三行注释去除，将 if 判断语句注释即可。

到达第一个点时机械臂状态如下：

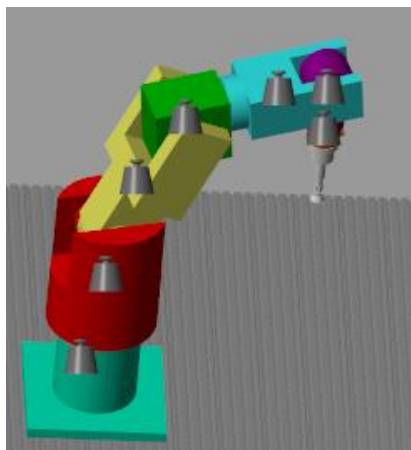


图 7.3 到达第一个点状态图

到达第二个点时机械臂状态如下：

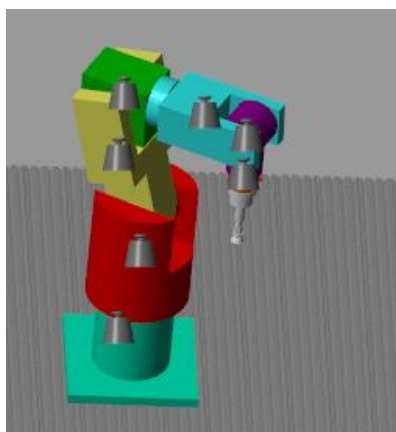


图 7.4 到达第二个点状态图

第八节 控制实物运行

用 RTSim Plus 中的 reset_auto 程序将机械臂复位至原位，然后将 teach_from_joint.slx 逆解程序、DH 参数以及轨迹模块设计等进行修改然后编译，编译成功后刷新 teach_from_joint 仿真配置，最后运行可得：



图 8.1 到达第一个点状态图



图 8.2 更改 $\text{pitch}=10^\circ$ 时



图 8.3 到达第二个点状态图

第三章 设计总结

在完成“FS003 机器人综合设计（半实物仿真 / 半实物控制）”课程设计的过程中，以垂直串联六关节机器人实验平台为核心，深入探索了机器人控制系统的开发与调试逻辑，系统实践了半实物仿真与半实物控制的关键技术。通过理论与实践的深度融合，不仅夯实了机器人学相关专业知识，更提升了工程实践能力与问题解决能力。现将本次课程设计的具体情况、收获与反思总结如下：

第一节 课程设计背景与目的

随着工业自动化技术的飞速发展，六关节机器人凭借其高度灵活性和空间作业能力，在智能制造、机械加工等领域得到广泛应用。本次课程设计依托六关节机器人实验平台，该平台为机器人控制系统开发调试提供了优质的半物理验证环境，能够支持轨迹规划、空间插补算法、正 / 逆运动学解算等核心理论算法的快速验证，为课程设计的开展提供了坚实的硬件与软件基础。

本次课程设计的核心目的在于：一是深入理解六关节机器人的系统组成与工作原理，掌握串联式开链结构的机械特性与传动逻辑；二是熟练运用 Matlab/Simulink 工具进行半实物仿真系统开发，实现运动学算法的编程与验证；三是掌握示教系统的操作与编程方法，完成机器人的半实物控制实践；四是培养工程设计思维，提升团队协作与问题解决能力，为后续专业学习与工程应用奠定基础。

第二节 课程设计主要内容与实施过程

2.1 机器人系统认知与前期准备

课程设计初期，首先对实验平台的机器人系统进行了全面认知。该系统由机器人本体、电控柜、快速原型仿真器和机器人控制软件组成。机器人本体为 6 自由度串联结构，各关节轴线相互平行或垂直，采用步进电机驱动，通过锥齿轮、同步齿型带等模块化传动结构实现运动传递，与工业机器人的伺服电机 + 谐波减速器 + 抱闸设计形成鲜明对比，其可视化的传动部件设计也为理解机械结构提供了便利。

电控柜内部集成了 2 套控制器、2 块电源等核心部件，整个控制系统通过控制器主板 BIOS 实现示教系统与仿真系统的切换。在前期准备阶段，我系统学习了机器人学的基础理论，重点梳理了 DH 参数建模、正 / 逆运动学解算、轨迹规划等核心知识点，同时熟悉了 VXWORKS 操作系统环境和 Matlab/Simulink 仿真工具的使用方法，为后续设计工作做好理论与技能铺垫。

2.2 半实物仿真系统设计与实现

半实物仿真系统的开发是本次课程设计的核心任务之一。该系统基于

Matlab 的 Simulink 模型进行开发，运动学算法完全开放，为自主编程与验证提供了灵活空间。在设计过程中，首先根据机器人的结构参数，建立了 DH 坐标系，通过查阅实验平台技术手册与测量数据，准确确定了各关节的连杆长度、扭转角等 DH 参数，完成了机器人的数学建模。

随后，运用 Matlab 的 M 脚本语言，编写了正运动学解算程序，通过输入各关节的角度值，求解末端执行器的空间位姿，并通过 Simulink 搭建仿真模型进行验证，确保解算结果的准确性。在逆运动学解算环节，针对六关节机器人逆解的复杂性，采用解析法进行求解，通过拆分机器人结构为肩部、肘部、腕部三个子系统，逐步推导各关节角度的数学表达式，最终实现了任意末端位姿下关节角度的求解，并通过仿真验证了算法的有效性。

此外，我还完成了轨迹规划算法的设计与仿真，采用关节空间插值法中的三次多项式插值，规划了机器人从初始位置到目标位置的运动轨迹，有效避免了关节运动的冲击与抖动，确保了运动的平稳性。通过 Simulink 仿真，直观观察了轨迹规划的效果，对插值参数进行了优化调整，使机器人运动速度与加速度满足设计要求。

3.3 半实物控制系统设计与调试

在半实物控制实践中，我重点掌握了示教系统的操作与编程方法。示教系统通过示教器进行编程，能够控制机器人按照设定的程序和速度进行运动。在调试过程中，首先熟悉了示教器的操作界面与功能按钮，学习了关节运动、线性运动等基本运动模式的设定方法，通过示教器手动控制机器人各关节运动，观察机器人在工作空间内的运动范围与灵活性。

随后，进行了程序编写与调试，根据预设的作业任务，编写了包含运动指令、速度指令、等待指令等的控制程序，实现了机器人从取料点到卸料点的自动作业流程。在调试过程中，遇到了机器人定位精度不足的问题，通过检查电机驱动器参数、调整运动速度与加速度等方式，逐步优化了控制程序，最终使机器人能够在工作空间内任意位置实现精确定位，满足作业要求。

同时，还对示教系统与仿真系统的切换逻辑进行了研究，通过控制器主板 BIOS 完成了两个系统的切换操作，对比分析了两个系统的特点与适用场景：仿真系统适用于理论算法的开发与验证，而示教系统适用于实际作业任务的编程与执行。

第三节 课程设计收获与体会

3.1 专业知识的深化与拓展

通过本次课程设计，我对六关节机器人的系统组成、机械结构与控制原理有了更为深入的理解。原本抽象的 DH 参数建模、正 / 逆运动学解算、轨迹规划

等理论知识，在实践中变得具体可感。不仅熟练掌握了这些核心算法的原理与实现方法，还对工业机器人与实验平台机器人的差异有了清晰的认识，了解了伺服电机与步进电机、谐波减速器与普通减速器的应用场景与性能特点，拓宽了专业知识面。

同时，在使用 **Matlab/Simulink** 进行仿真开发和 **VXWORKS** 环境下的控制编程过程中，我的软件操作能力与编程水平得到了显著提升，学会了将理论算法转化为实际代码，并通过仿真工具进行验证与优化。

3.2 工程实践能力的提升

本次课程设计是一次完整的工程设计实践，从系统认知、方案设计、编程实现到调试优化，每个环节都需要严谨的逻辑思维与扎实的实践能力。在面对逆运动学解算困难、仿真与实际系统偏差等问题时，我学会了冷静分析问题根源，通过查阅资料、请教老师、团队讨论等方式寻找解决方法，逐步培养了发现问题、分析问题与解决问题的能力。

此外，在程序编写与调试过程中，深刻体会到了工程设计的严谨性，一个微小的参数错误或逻辑漏洞都可能导致整个系统无法正常运行。这让我养成了注重细节、反复验证的良好习惯，提升了工程实践的规范性与可靠性。